

Department of Computer Science and Engineering

1. Subject Code: **CPT-501** Course Title: **Programming Methodologies - II**
2. **Expected Effort:** Total: 150 Hours Pacing: 10 Hours / Week Duration: 15 Weeks
3. **Prerequisites**
 - CPT-301
 - SES-401
4. **Course Structure**
5 Units $\times$ 30 Hours (lectures, source-code reading, laboratory, PoW)

1. Course Description

CPT-501 extends the pattern vocabulary of CPT-301 into the algorithmic machinery required for research, contest and large-scale systems work. Students implement, benchmark and reuse a personal library of advanced structures — flows, segment trees, suffix automata, sketches — and are assessed under contest conditions as well as on engineering quality. The final unit deliberately connects classical algorithm design to the probabilistic and streaming algorithms that appear inside the databases, caches and retrieval systems studied.

2. Course Outcomes

After successful completion of the course, students will be able to:

- CO1.** Model and solve constrained assignment, connectivity and routing problems using advanced graph algorithms and network flows.
- CO2.** Design efficient solutions using advanced dynamic programming paradigms and DP optimisation techniques.
- CO3.** Implement and reason about range-query, persistent and amortised data structures with provable complexity bounds.
- CO4.** Apply string algorithms, number-theoretic methods and computational geometry primitives to large inputs.
- CO5.** Analyse randomised, approximate and streaming algorithms, and execute contest-grade problem solving under time pressure.

3. Detailed Syllabus

Unit	Contents	Hrs
1	Advanced Graph Algorithms & Network Flows <i>Module A: Shortest Paths and Spanning Structures</i> Dijkstra with binary and Fibonacci-style heaps, 0–1 BFS, Bellman–Ford and negative cycle detection, Floyd–Warshall, Johnson’s algorithm. Minimum spanning trees (Kruskal, Prim), second-best MST. <i>Module B: Connectivity and Decomposition</i> Tarjan and Kosaraju strongly connected components, condensation graphs, bridges and articulation points, biconnected components, 2-SAT via implication graphs, Euler tours. <i>Module C: Flows and Matching</i> Max-Flow Min-Cut theorem, Edmonds–Karp and Dinic’s algorithm, scaling variants. Bipartite matching and König’s theorem, Hopcroft–Karp. Min-cost max-flow and assignment problems. Modelling reductions from scheduling and selection problems. Hands-on Laboratory & Proof-of-Work (PoW) • PoW: Build a shift-assignment engine that reduces a staffing constraint problem to min-cost max-flow and validates optimality against a brute-force oracle. • PoW: Implement an SCC-based build dependency resolver that reports cyclic components with minimal breaking edges. • Benchmark Dinic versus Edmonds–Karp on dense and sparse generated graphs.	30
2	Advanced Dynamic Programming & DP Optimisation <i>Module A: Structured DP States</i> Bitmask DP over subsets, DP on trees (rerooting technique), DP on DAGs, digit DP, interval DP, probability and expectation DP, game theory DP (Sprague–Grundy). <i>Module B: DP Optimisation Techniques</i> Convex Hull Trick and Li Chao trees, divide-and-conquer optimisation, Knuth optimisation, monotone queue optimisation, sum-over-subsets DP. Recognising when each transformation is admissible. <i>Module C: Algebraic Acceleration</i> Matrix exponentiation for linear recurrences, binary lifting, sparse ancestor queries, and meet-in-the-middle search. Hands-on Laboratory & Proof-of-Work (PoW) • PoW: Implement a bitmask travelling-salesman solver and compare it with a held-karp-plus-pruning variant on generated instances. • PoW: Reduce an $O(n^2)$ DP to $O(n \log n)$ using the Convex Hull Trick and evidence the asymptotic change empirically. • Implement binary lifting for lowest common ancestor queries on a million-node tree.	30

Unit	Contents (continued)	Hrs
3	Range Queries, Advanced Trees & Amortised Structures <i>Module A: Range Query Structures</i> Fenwick trees (1D and 2D), Segment Trees with lazy propagation, merge-sort trees, Sparse Tables and idempotent queries, square-root decomposition, Mo's algorithm for offline queries. <i>Module B: Tree Decomposition and Persistence</i> Heavy-Light Decomposition, Euler tour + segment tree, centroid decomposition, small-to-large merging. Persistent arrays and persistent segment trees; versioned query semantics. <i>Module C: Balanced and Amortised Structures</i> Treaps and randomised balancing, order-statistic trees, DSU with roll-back and offline dynamic connectivity, amortised analysis via the potential method. Hands-on Laboratory & Proof-of-Work (PoW) • PoW: Ship a tested, documented segment-tree library supporting lazy range updates, persistence and custom monoid merge functions. • PoW: Build a range-analytics service answering 10^5 mixed update/query operations within a fixed latency budget. • Solve an offline query set with Mo's algorithm and compare against an online structure.	30
4	Strings, Number Theory & Computational Geometry <i>Module A: String Algorithms</i> KMP failure function, Z-algorithm, Rabin–Karp hashing and anti-hash considerations, Aho–Corasick automata, suffix arrays with LCP (Kasai), suffix automaton basics, Manacher's algorithm. <i>Module B: Number Theory and Combinatorics</i> Modular arithmetic, fast exponentiation, extended Euclid and modular inverse, sieve variants, factorisation, Chinese Remainder Theorem, Euler totient. Combinatorial counting, inclusion–exclusion, Möbius inversion. FFT/NTT for convolution. <i>Module C: Computational Geometry</i> Vector primitives, orientation predicates and numerical robustness, convex hull (Andrew monotone chain), line and segment intersection, sweep line algorithms, closest pair, polygon area and point-in-polygon. Hands-on Laboratory & Proof-of-Work (PoW) • PoW: Build a multi-pattern plagiarism detector using Aho–Corasick and suffix arrays, benchmarked on a corpus of at least 100 MB. • PoW: Implement a sweep-line collision detector over generated geometric workloads with exactness tests. • Implement NTT-based polynomial multiplication and validate against a naive convolution.	30

Unit	Contents (continued)	Hrs
5	Randomised, Approximate & System-Scale Algorithms with Contest Execution <i>Module A: Randomisation and Approximation</i> Monte Carlo versus Las Vegas algorithms, reservoir sampling, randomised quickselect, hashing with universal families. NP-hardness intuition, approximation ratios, local search and simulated annealing heuristics. <i>Module B: Streaming and External-Memory Algorithms</i> Bloom filters and counting Bloom filters, Count–Min Sketch, HyperLogLog cardinality estimation, top-k heavy hitters, reservoir and weighted sampling. Cache-aware and cache-oblivious analysis; B-tree and LSM-tree algorithmic behaviour; consistent hashing and rendezvous hashing. <i>Module C: Contest and Engineering Protocol</i> Contest strategy and time allocation, stress testing with random generators and brute-force oracles, invariant assertions, edge-case taxonomies, upsolving discipline, and written editorial production. Hands-on Laboratory & Proof-of-Work (PoW) • PoW: Implement a sketching library (Bloom filter, Count–Min Sketch, HyperLogLog) and report measured error bounds against exact counting at several memory budgets. • PoW: Compete in a series of rated contests and submit an upsolving log with written editorials for every unsolved problem. • Build a stress-testing harness that automatically minimises failing test cases.	30
Total Instructional Effort		150

4. Recommended Resources

- Cormen et al., *Introduction to Algorithms*, 4th ed. (Parts VI–VIII).
- Laaksonen, *Competitive Programmer’s Handbook*; Halim, *Competitive Programming 4*.
- Sedgewick & Flajolet, *An Introduction to the Analysis of Algorithms*.
- Motwani & Raghavan, *Randomized Algorithms*.
- CP-Algorithms reference site; USACO Guide advanced modules.